

Frontend Take Home Evaluation

評估目的

本評估希望了解候選人在現代 Web Engineering 環境中的：

- 架構與可維護性思維
- Production / Delivery Thinking
- AI / Agent 協作與工程判斷能力

我們更重視：

- 問題拆解 / 系統演進能力
- Tradeoff Thinking / Delivery Quality

而不只是功能完成度。

時間限制

Phase 1 — Design / Planning (約 1 小時)

可與 AI / Agents 討論：

- 架構方向
- Workflow Planning
- Implementation Strategy

建議保留設計筆記、Tradeoff 分析與 AI / Agent 協作紀錄，作為後續 On-site Discussion 的一部分。

Phase 2 — Implementation (約 1~2 小時)

開始與 AI / Agents 協作開發。

時間將以：

- First Commit 至最後 Commit
- 或第一個 Commit 後約 120 分鐘內的 Commits

作為主要參考。

請在有限時間內做出合理取捨，並說明哪些部分優先、哪些部分刻意不做。

若時間不足，可優先完成核心架構與設計方向。

題目（二選一）

A. Mocking momoshop

參考並建立 momo 電商網站（純前端實作，無任何真實網站 API 呼叫）。

建議路由（至少一個）

- `/, /search/...`
- `/goods/..., /discover/..., /live/...`

可自由決定：

- Component Architecture / Routing
- State Management / Rendering Strategy / Mock Data Structure

可自行設計並使用 Mock Data，不需串接真實後端。

考核重點

- 實作結果與真實網站的差異
- 開發流程規劃與 Agent 協作效率評估/分析

Bonus

- Human-Agent Iteration Architecture
 - Validation / Observability Thinking
-

B. Merchant Card Showroom

分析真實 momo 電商商品卡類型，建立 Card Showroom。

基本需求（至少一個卡片）

- 列出所有商品卡 / 展示單一商品卡
- 商品卡細節調整介面 - Browser-side Persistence
- 提供 sample html 使用上述商品卡的範例（可以是 web component 或script載入的方式）

Bonus

- Reusable Card Architecture
 - Schema / Plugin Extensibility
 - State Consistency Strategy
-

技術限制

不限框架與工具。可自由使用：

- Codex / Claude / Gemini / Cursor / Copilot ...
- OSS Libraries

我們更重視：

- Engineering Judgment / Maintainability
- AI / Agent Supervision Capability

不需追求完整商業功能或 UI 精緻度，我們更重視系統設計、可維護性與工程判斷。

提交內容

請提供：

- Source Code (GitHub Repo 或 Zip with Git History)
- Git History (包含 Agent Co-authored Logs)
- README / Docs / 架構與 Tradeoff 說明 / 後續演進方向

README 與 Commit History 也將作為評估的一部分。請將自己視為：

此系統的長期維護者，而不只是功能實作者。